

Missouri University of Science and Technology

Intro to Data Science
CS 3402 (Spring 2026)

Lecture 17 CNN & ResNet

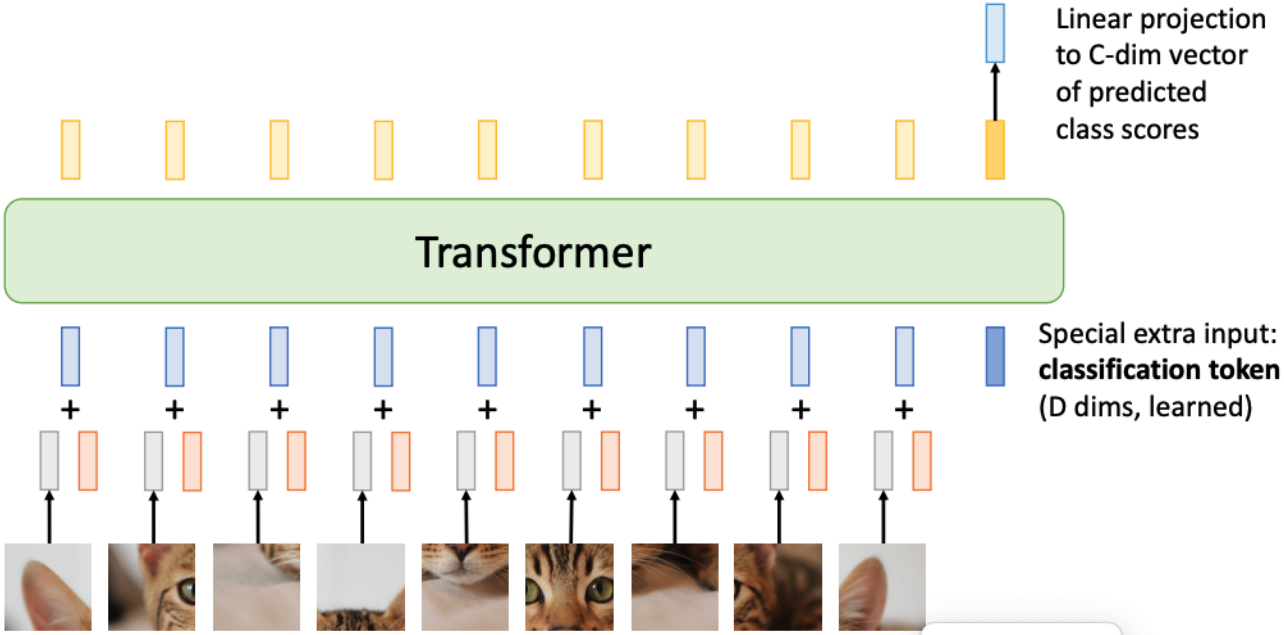
Xiaowei Yu, Ph.D.

Department of Computer Science

Last class: Vision Transformer

How can Transformer process images?

- ✔ Tokenization
- ✔ Input Embeddings
- ✔ Position Encodings
- ✔ Query, Key, & Value
- ✔ Attention
- ✔ Self Attention
- ✔ Multi-Head Attention
- ✔ Feed Forward
- ✔ Add & Norm
- ✔ Encoders

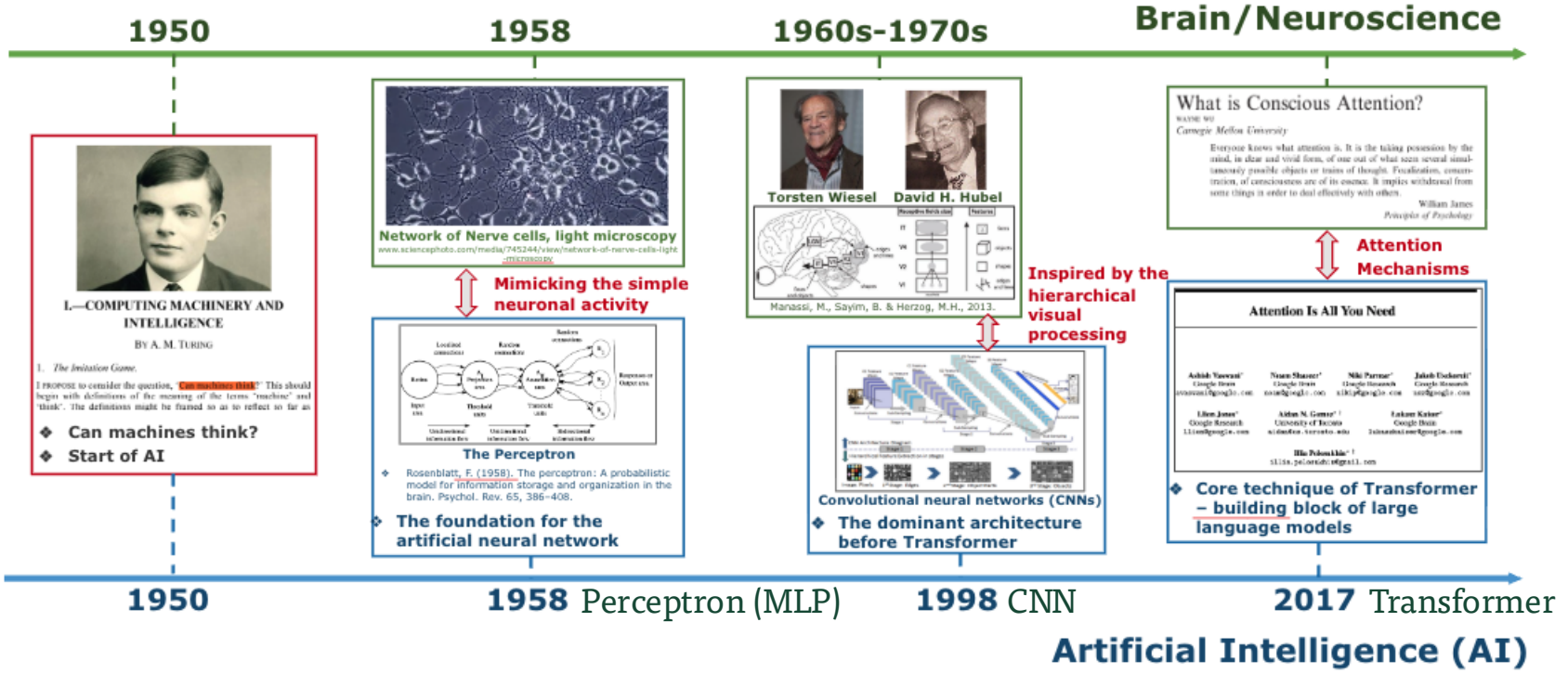


An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, ICLR 2021

Today's Topics

1. From Human Vision to Computer Vision
2. CNNs – Convolutional Layers
3. CNNs – Pooling Layers
4. Introduce to Homework2

From Human Vision to Computer Vision



From Human Vision to Computer Vision

What do you see?



From Human Vision to Computer Vision

How do you see?

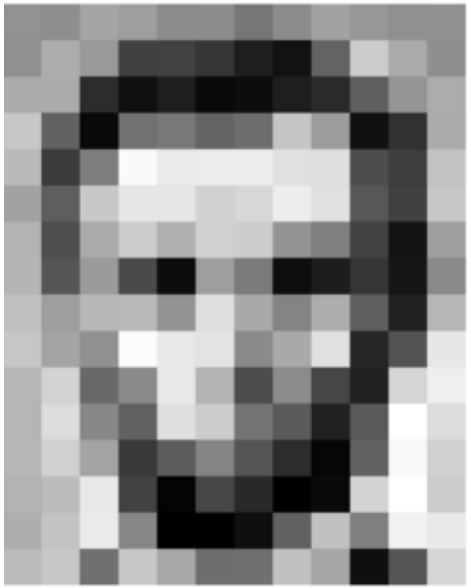




How can we help computers see?

From Human Vision to Computer Vision

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

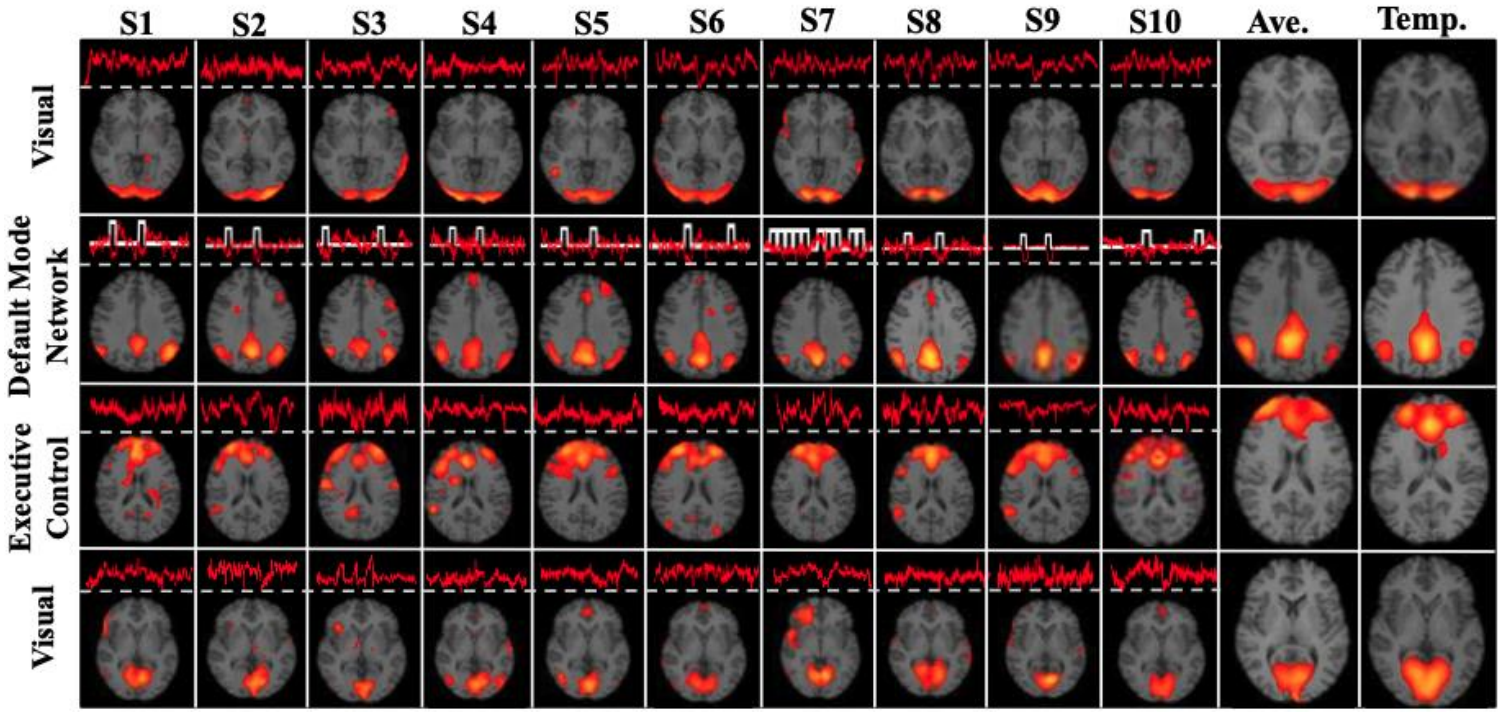
What the computer "sees"

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

What computers 'see': Images as Numbers

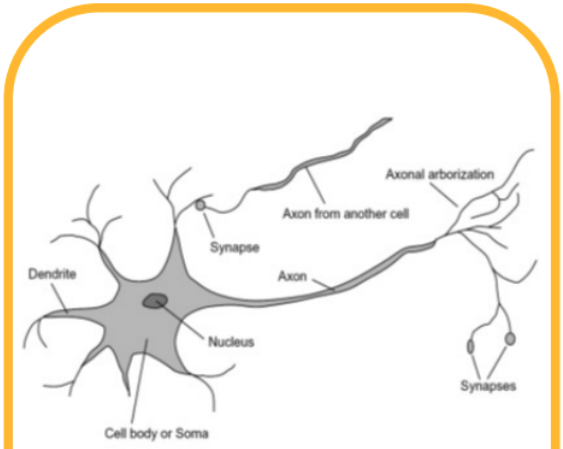
From Human Vision to Computer Vision



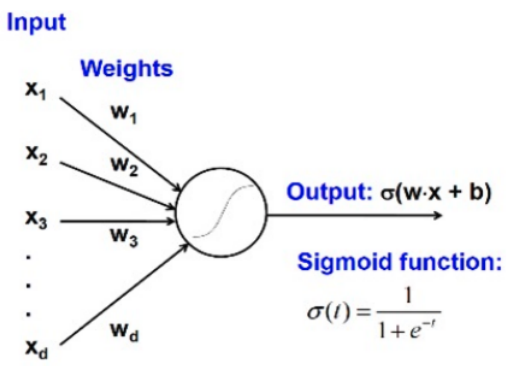
Activated brain networks for visual information processing.

From Human Vision to Computer Vision

Recall:



A biological neuron



An artificial neuron (Perceptron)
- a linear classifier

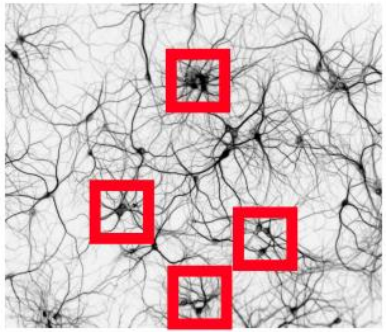


Slide credit: Jia-Bin Huang

From Human Vision to Computer Vision

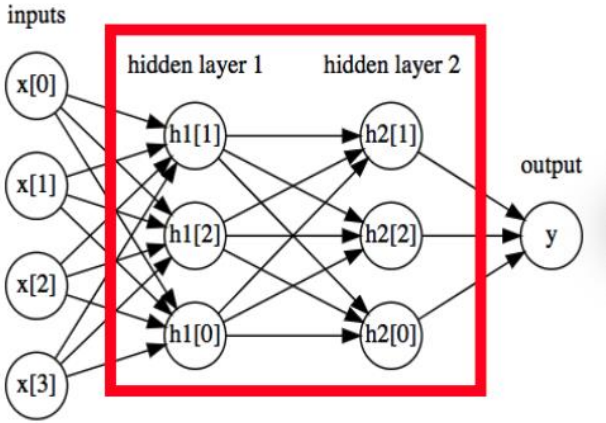
Recall:

Biological Neural Network:



http://www.rzagabe.com/2014/11/03/an-introduction-to-artificial-neural-networks.html

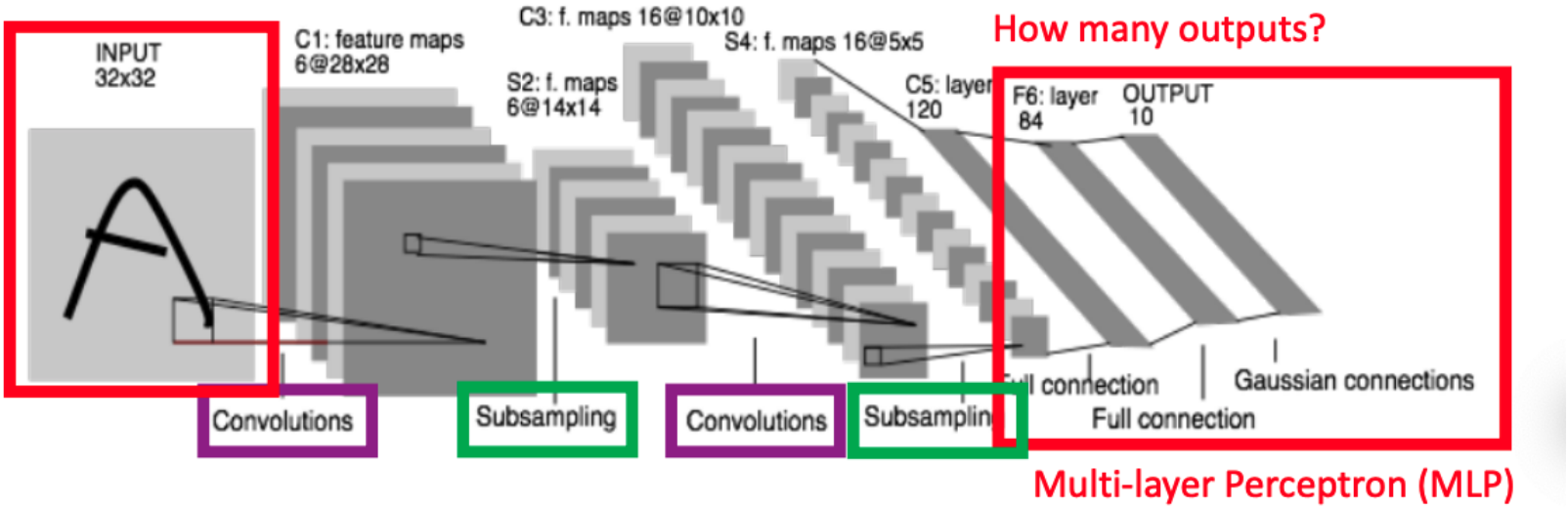
Artificial Neural Network:



https://github.com/amueller/introduction_to_ml_with_python/blob/master/02-supervised-learning.ipynb

Convolutional Neural Networks (CNN)

CNN: Modeling Vision System



Slide Credit: https://people.eecs.berkeley.edu/~jrs/189/lec/cnn.pdf

Y. Lecun ; L. Bottou ; Y. Bengio ; P. Haffner; Gradient-based learning applied to document recognition; 1998

Convolutional Neural Networks (CNN)

CNN: Catalyst for Computer Vision Industry Boom



Self-driving cars



Self-driving vehicle on Mars



Guided surgery

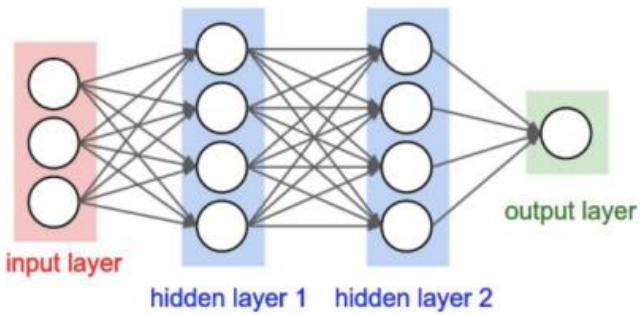


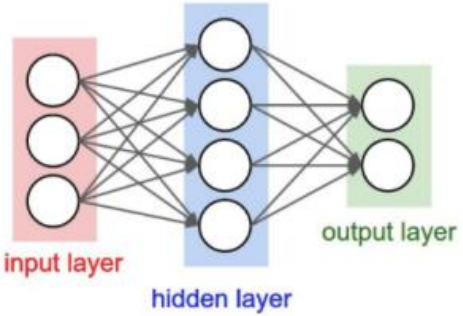
Visual assistance for people who are blind



Security

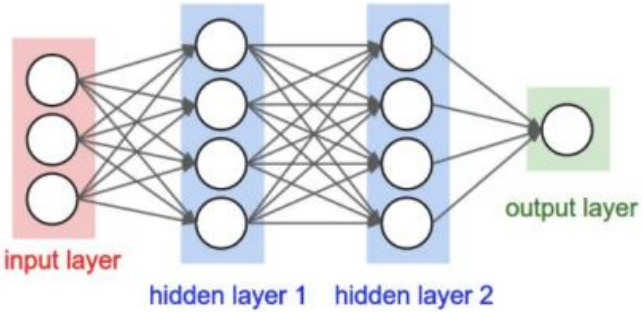
Recall: Fully-Connected Neural Networks

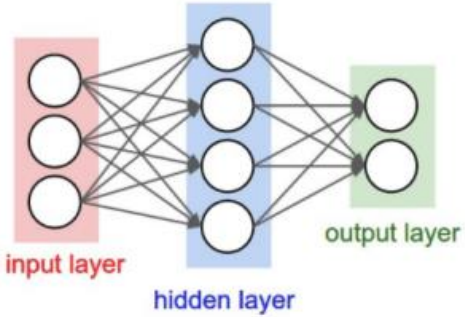




Each node provides input to each node in the next layer

Recall: Fully-Connected Neural Networks

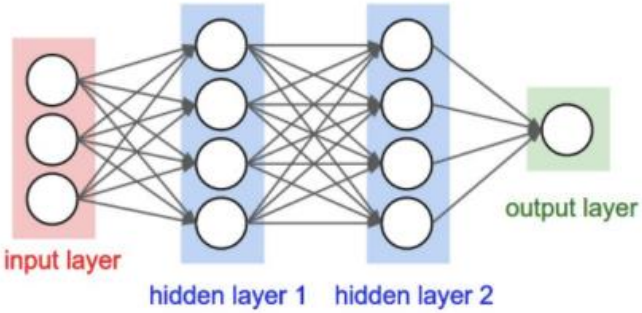


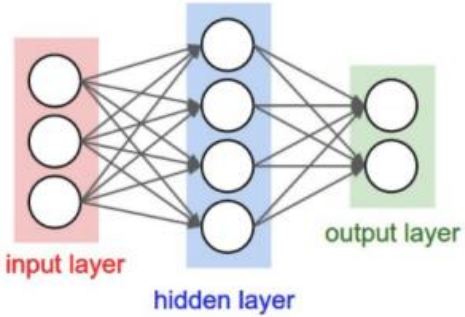


What is a weakness of fully-connected networks?

- Assume 2 layer model with 100 nodes per layer
  - e.g., how many weights are in a 640x480 image?
    - 640x480x3x100 + 100x100 + 100x1 = 92,170,100
  - e.g., how many weights are in a 2048X1536 image (3.1 Megapixel image)?
    - 2048x1536x3x100 + 100x100 + 100x1 = 943,728,500

Recall: Fully-Connected Neural Networks





Many model parameters in fully connected networks
Many model parameters means

- greater chance to overfit
- increased training time
- needs more training data

Today's Topics

1. From Human Vision to Computer Vision
2. CNNs – Convolutional Layers
3. CNNs – Pooling Layers
4. Introduce to Homework2

Convolutional Neural Networks (CNN)

Convolutional Layers

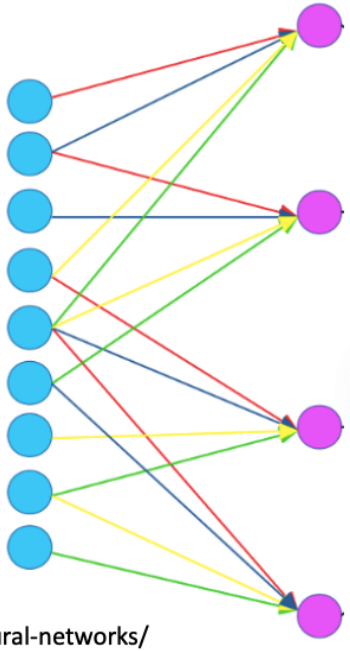
Idea: each node receives input only from a small neighborhood in previous layer

Convolutional Neural Networks (CNN)

Convolutional Layers

Convolutional Layer: Parameters to Learn

- For example, how many weights must be learned?
  - 4 (red, blue, yellow, and green values in the filter)
- If we instead used a fully connected layer, how many weights would need to be learned?
  - 36 (9 blue x 4 magenta)
- For example, how many parameters must be learned?
  - 5 (4 weights + 1 bias)
- If we instead used a fully connected layer, how many parameters would need to be learned?
  - 40 (36 weights + 4 bias)



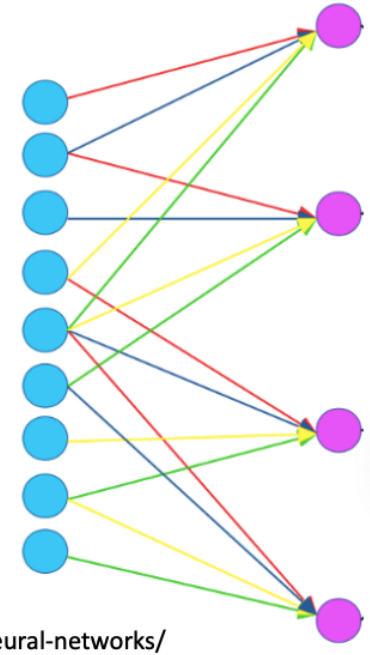
https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

Convolutional Neural Networks (CNN)

Convolutional Layers

Convolutional Layer: Parameters to Learn

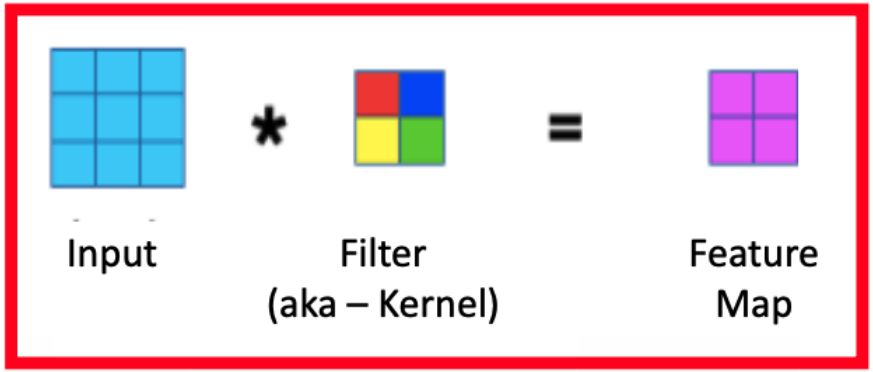
- Parameter sharing significantly reduces number of weights to learn and so storage requirements
- Sparse (rather than full) connectivity also significantly reduces the number of computational operations required



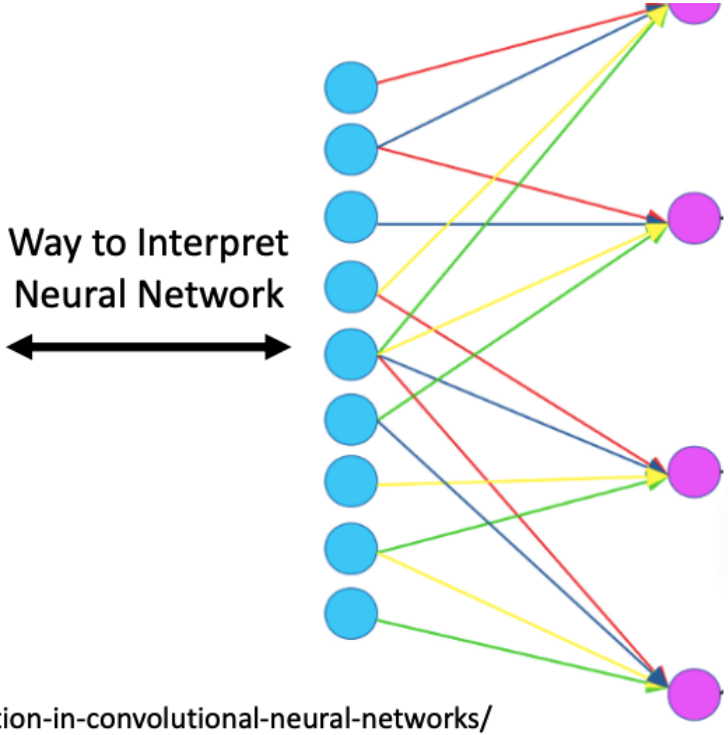
https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

Convolutional Neural Networks (CNN)

Convolutional Layer: Applies Linear Filter

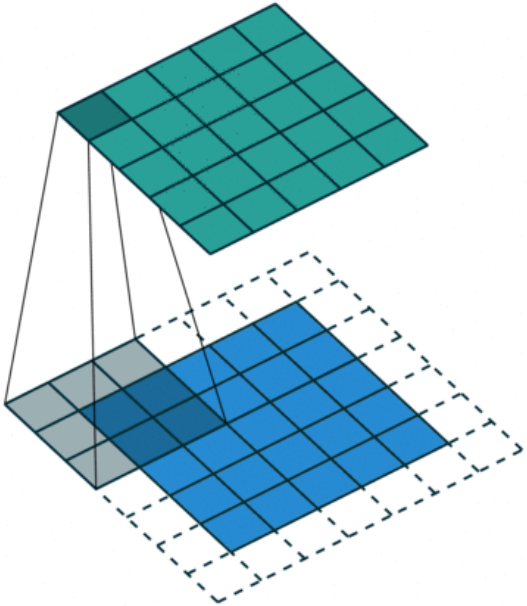


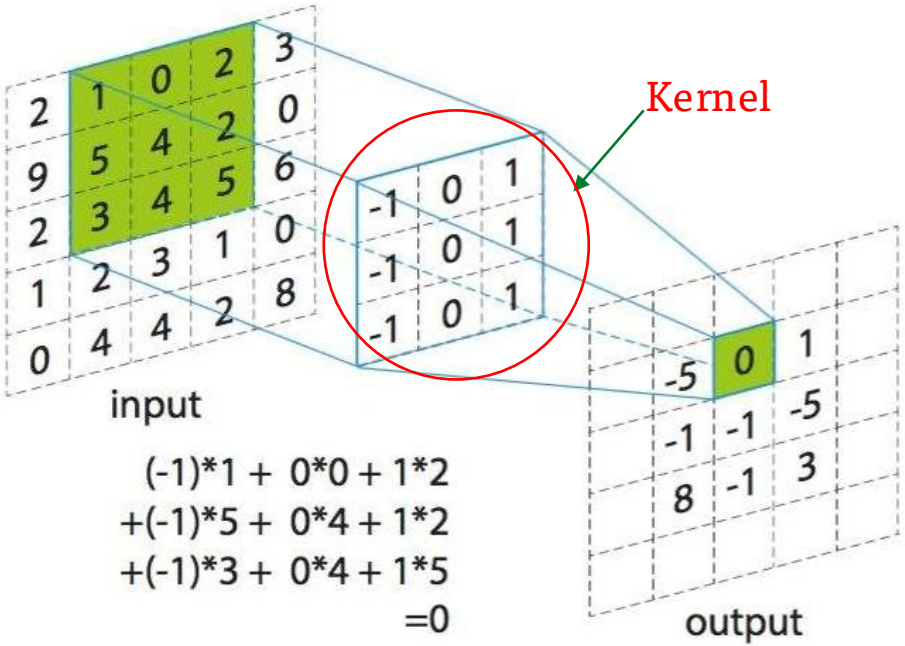
Way to Interpret
Neural Network



https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

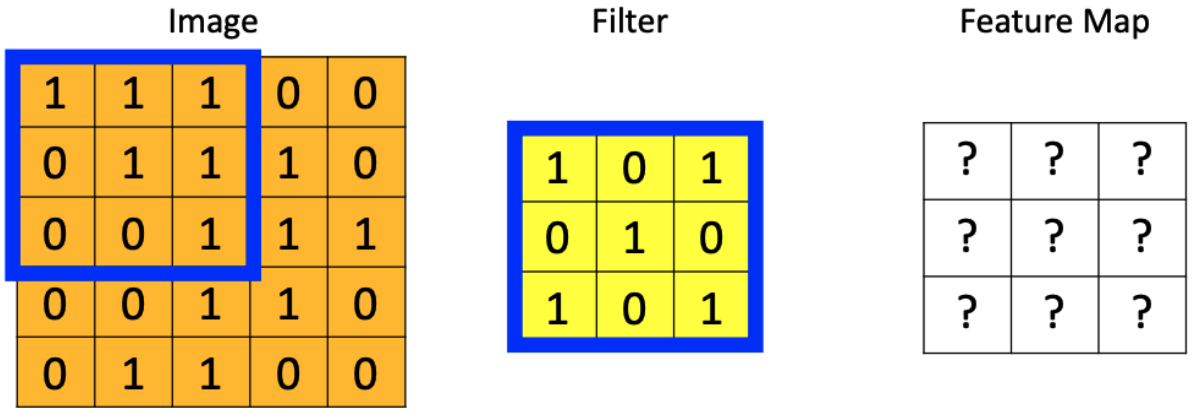
Convolutional Neural Networks (CNN)





Convolutional Neural Networks (CNN)

Image Filtering: Toy Example



Dot Product = 1*1 + 1*0 + 1*1 + 0*0 + 1*1 + 1*0 + 0*1 + 0*1 + 0*0 + 0*0 + 1*1

Dot Product = 4

Convolutional Neural Networks (CNN)

Stride = 1

Stride is a hyperparameter that dictates how many pixels a convolutional filter (kernel) shifts over the input data during each step.

Image

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter

1	0	1
0	1	0
1	0	1

Feature Map

4	?	?
?	?	?
?	?	?

Convolutional Neural Networks (CNN)

Stride = 1

Stride is a hyperparameter that dictates how many pixels a convolutional filter (kernel) shifts over the input data during each step.

Image

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter

1	0	1
0	1	0
1	0	1

Feature Map

4	3	?
?	?	?
?	?	?

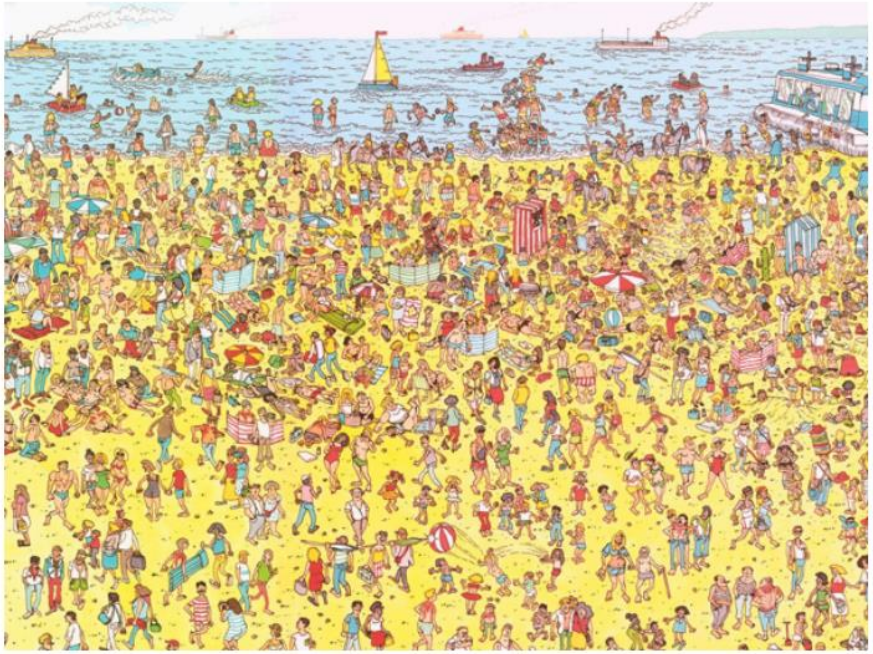
Feature map size = f(Image size N, filter size F, stride S) = ?

Convolutional Neural Networks (CNN)

Image Filter: What Does It Do? (Where's Waldo?)

Filter





Convolutional Neural Networks (CNN)

Image Filter: What Does It Do?

- e.g.,

Filter

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Visualization of Filter

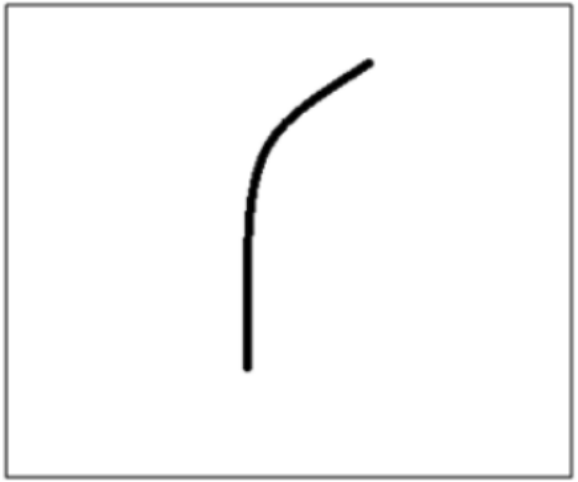


Image Credit: https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/

Convolutional Neural Networks (CNN)

Image Filter: What Does It Do?

- e.g.,

Filter Overlaid on Image



Image

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

*

Filter

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Weighted Sum = ?

Weighted Sum = (50x30) + (20x30) + (50x30) + (50x3) + (50x30)

Weighted Sum = 6600 **(Large Number!!)**

Image Credit: https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/

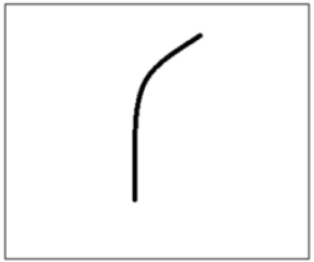
Convolutional Neural Networks (CNN)

Image Filter: What Does It Do?

- e.g.,

This Filter is a Curve Detector!

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0



Filter Overlaid on Image (Big Response!)



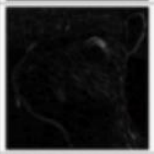
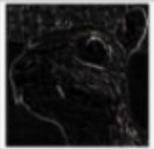
Filter Overlaid on Image (Small Response!)



Image Credit: https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/

Convolutional Neural Networks (CNN)

Different Filters Detect Different Features

	Filter	Feature Map
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

	Filter	Feature Map
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/

Convolutional Neural Networks (CNN)

1. How would you design a linear filter to “brighten” an image



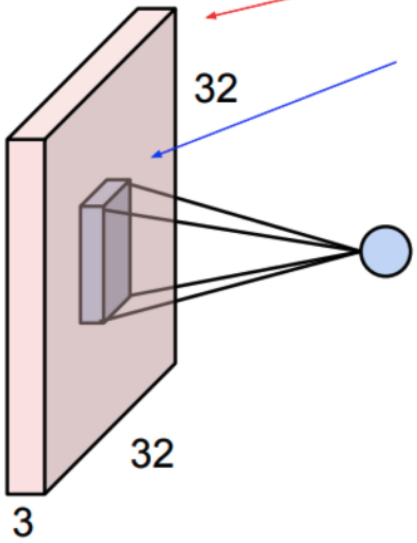
2. How would you design a linear filter to remove wrinkles/blemishes?





Convolutional Neural Networks (CNN)

Convolutional Layer: Introduce Non-Linearity



32x32x3 image

5x5x3 filter w

Apply activation function to number; e.g., ReLU

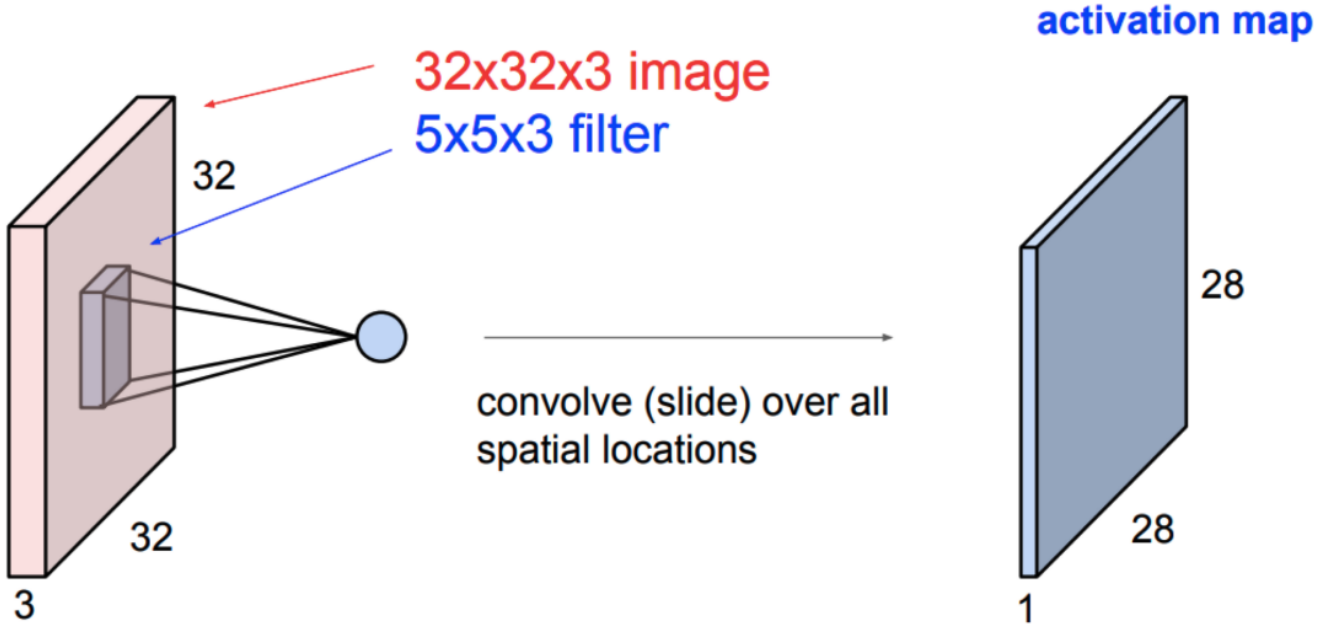
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

Slide Credit: http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Neural Networks (CNN)

Convolutional Layer

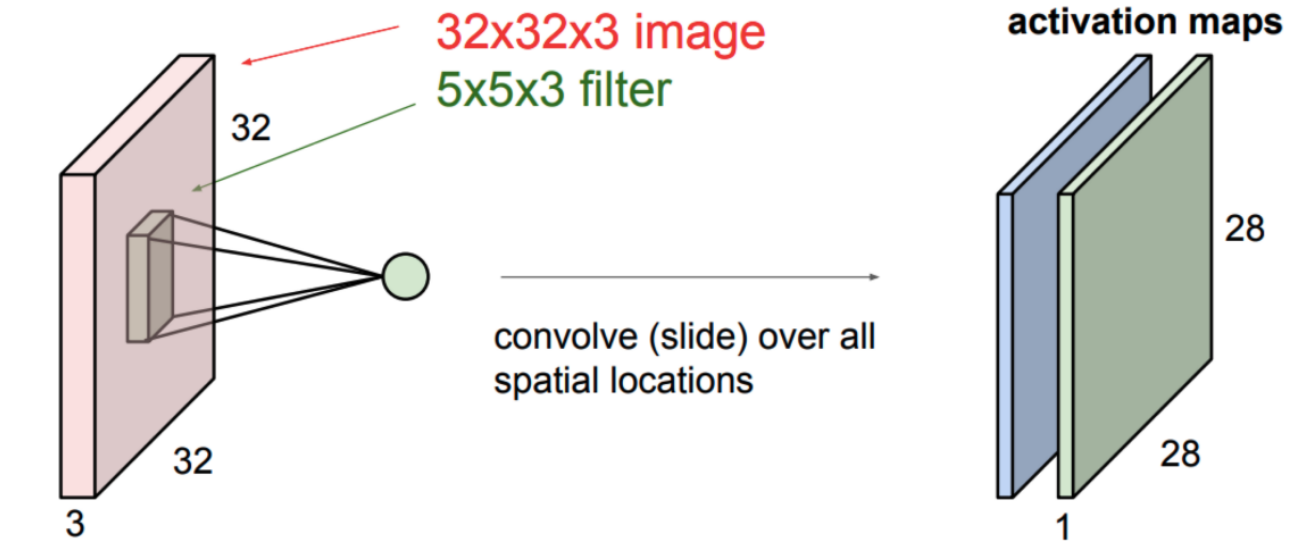


Slide Credit: http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Neural Networks (CNN)

consider a second, green filter

Convolutional Layer



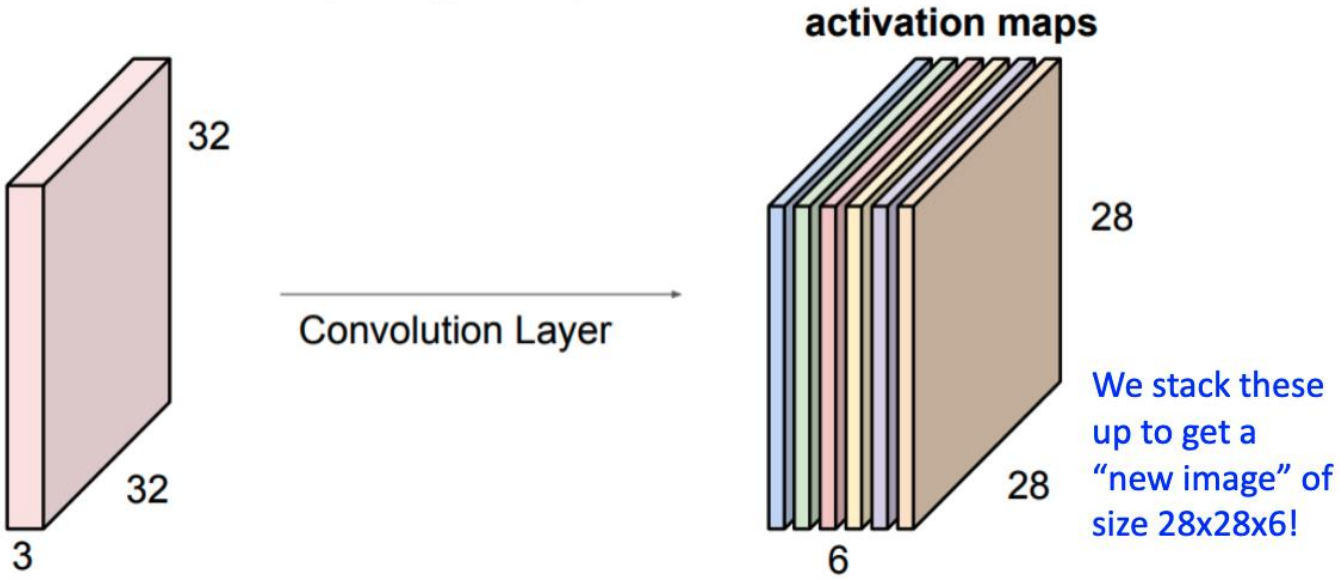
Slide Credit: http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Neural Networks (CNN)

Parameters: bank of filters and biases used to create the activation maps (aka – feature maps)

Convolutional Layer

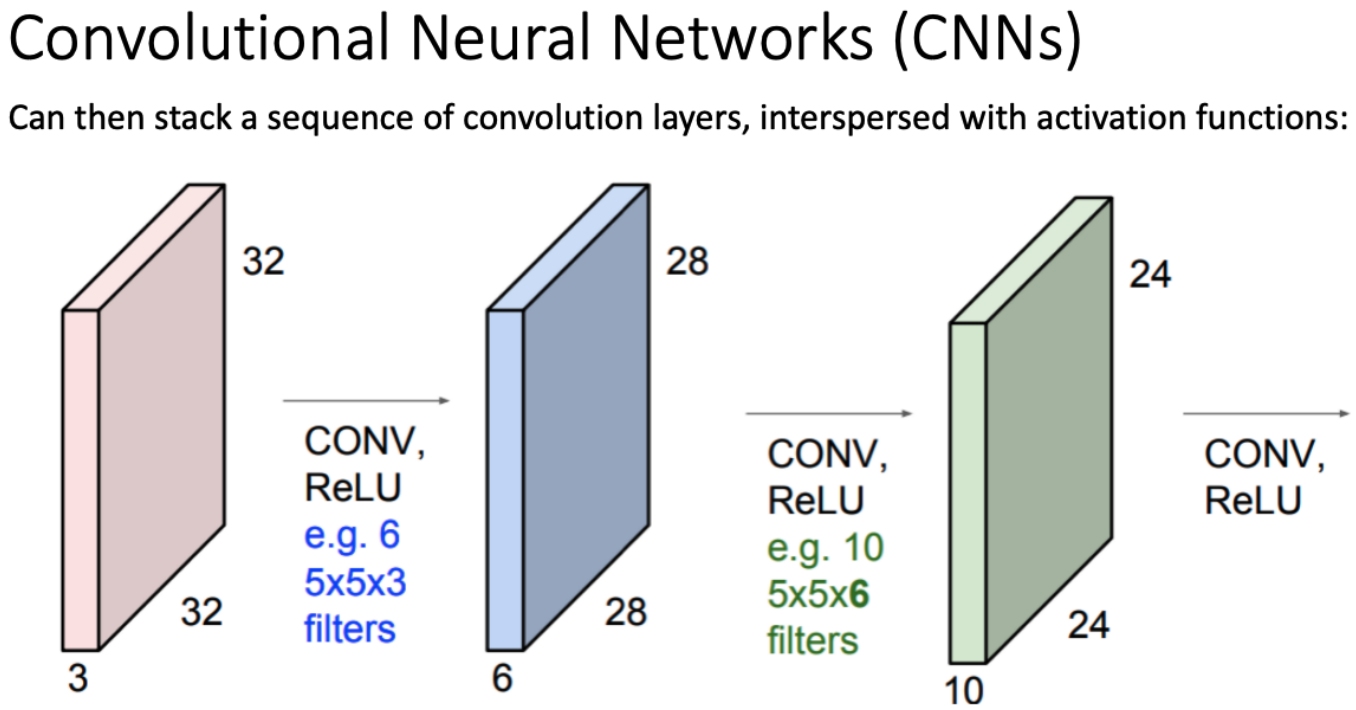
if we had 6 5x5 filters, we'll get 6 separate activation maps:



Slide Credit: http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

Convolutional Neural Networks (CNN)

Parameters: bank of filters and biases used to create the activation maps (aka – feature maps)



Slide Credit: http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf

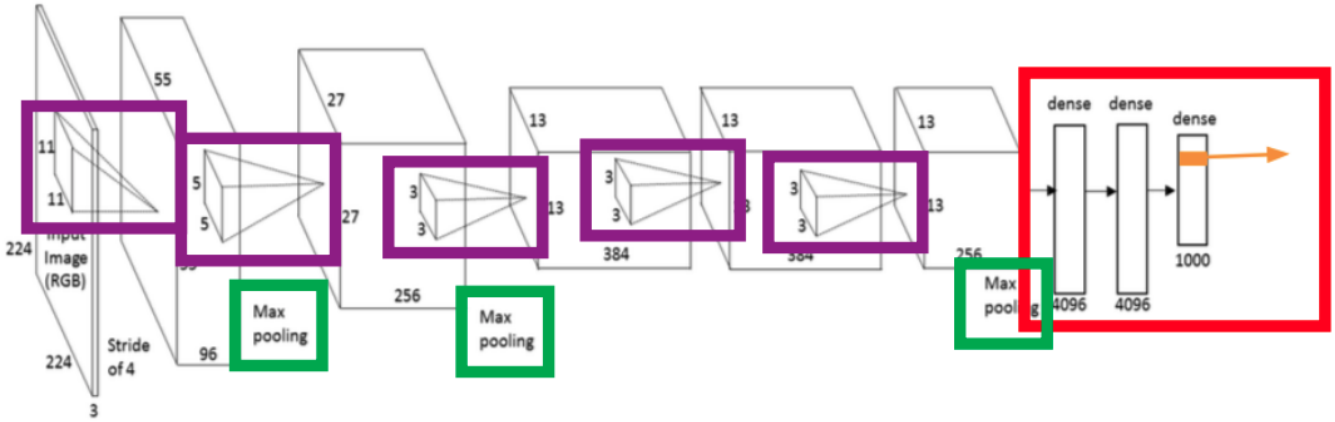
Today's Topics

1. From Human Vision to Computer Vision
2. CNNs – Convolutional Layers
3. CNNs – Pooling Layers
4. Introduce to Homework2

Convolutional Neural Networks (CNN)

CNN: Pooling Layers

- AlexNet extracts useful features of lower dimension prior to passing it to MLP with:
  - Convolutional layers
  - Pooling Layers



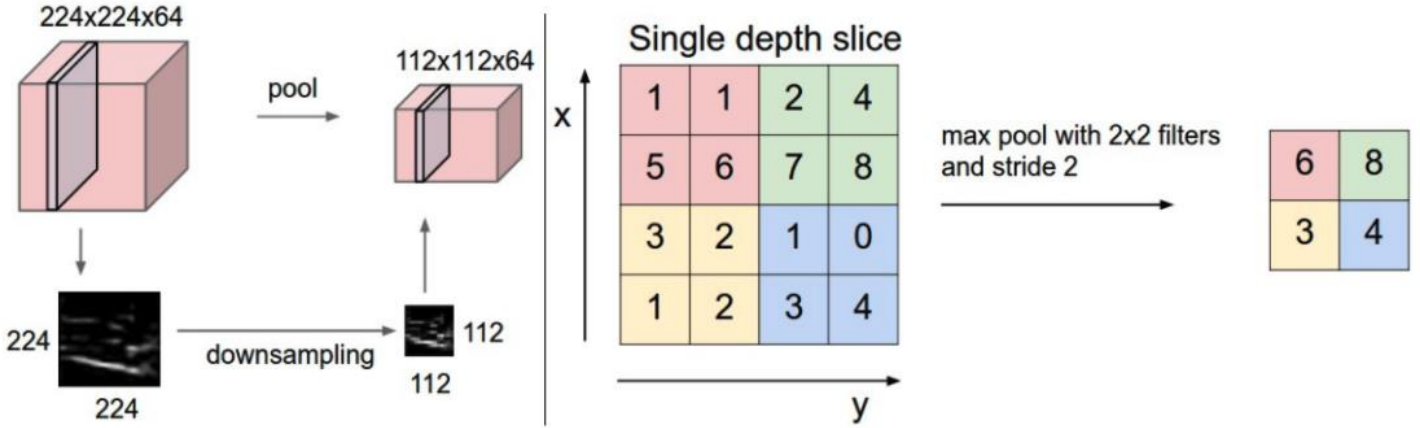
Slide Credit: https://www.slideshare.net/xavigiro/saliency-prediction-using-deep-learning-techniques

A. Krizhevsky, I. Sutskever, G. E. Hinton "ImageNet classification with deep convolutional neural networks"

Convolutional Neural Networks (CNN)

Pooling Layer: Summarizes Neighborhood

- **Max-pooling**: partitions input into a set of non-overlapping rectangles and outputs the maximum value for each chunk

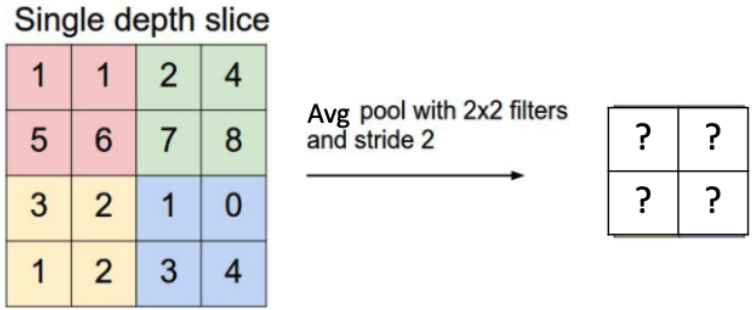


http://cs231n.github.io/convolutional-networks/#pool

Convolutional Neural Networks (CNN)

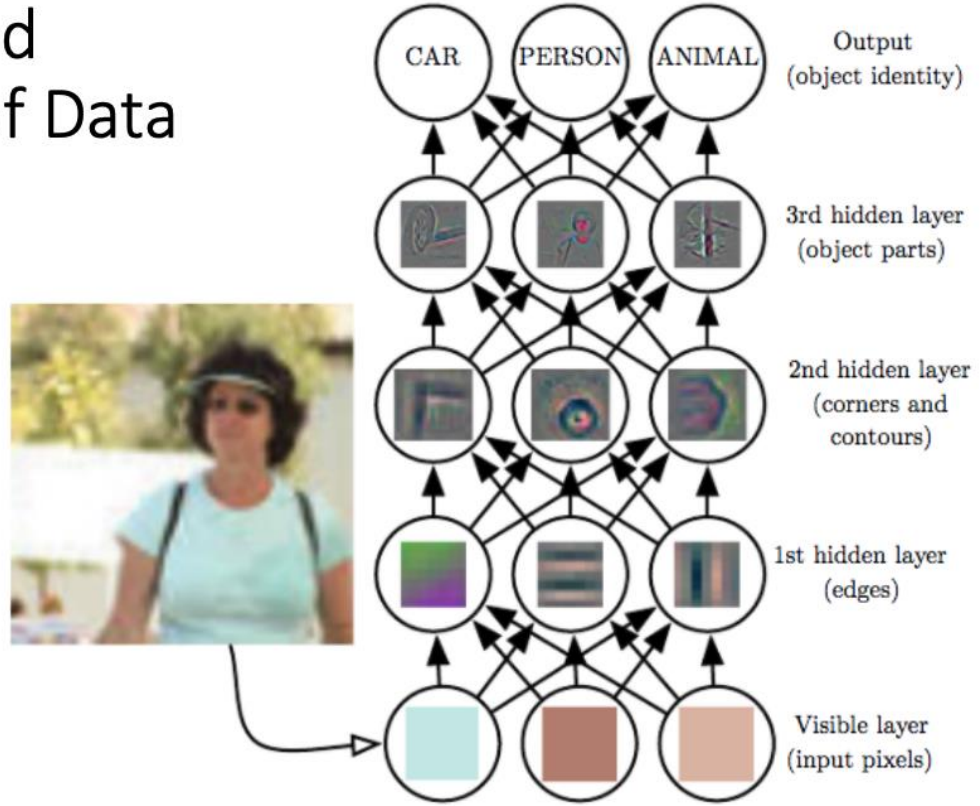
Pooling Layer: Summarizes Neighborhood

- **Max-pooling**: partitions input into a set of non-overlapping rectangles and outputs the maximum value for each chunk
- **Average-pooling**: partitions input into a set of non-overlapping rectangles and outputs the average value for each chunk



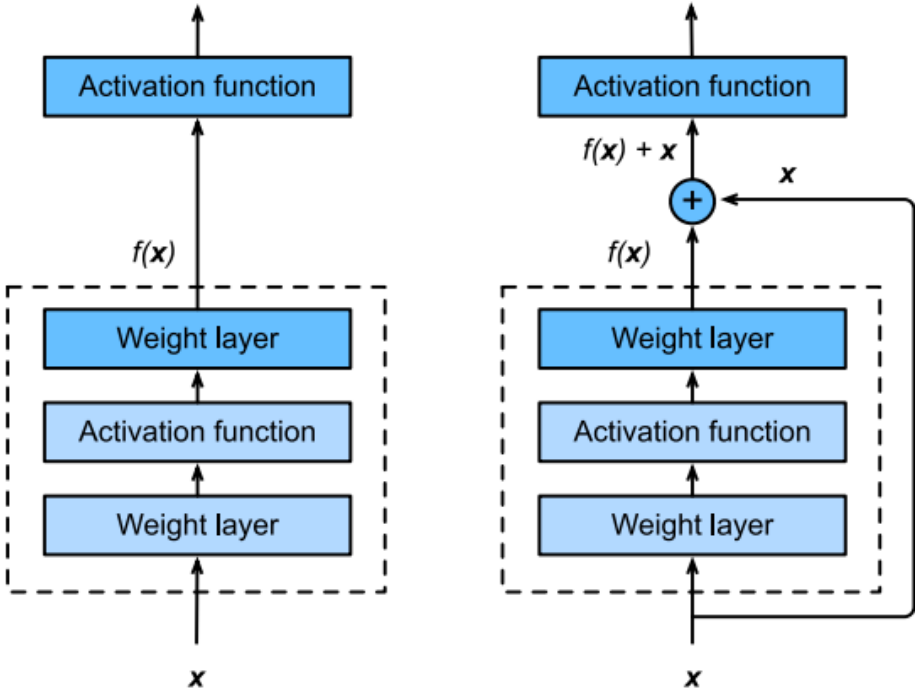
http://cs231n.github.io/convolutional-networks/#pool

CNN: Learns Good Representation of Data



Deep Learning, Ian Goodfellow, Yoshua Bengio, and Aaron Courville

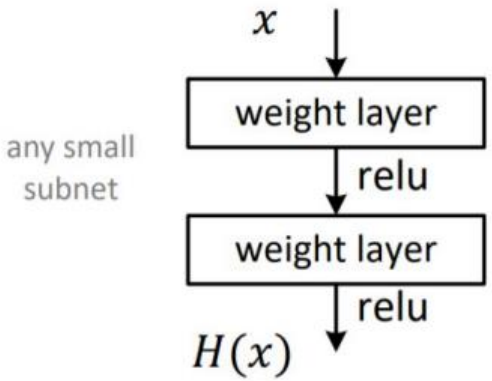
Residual Networks



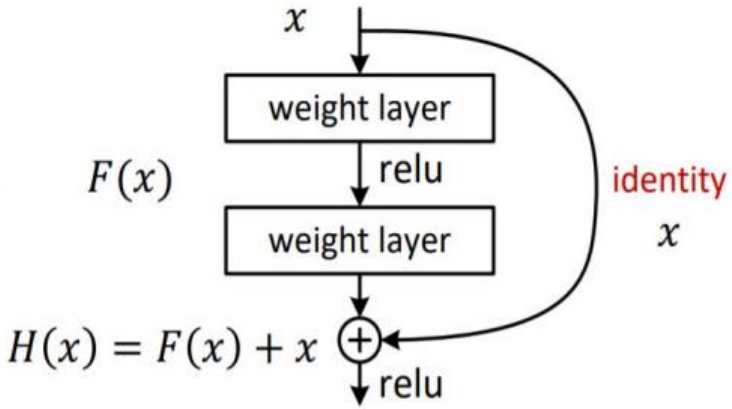
ResNet Block

Deep Residual Learning

- Plain net



- Residual net

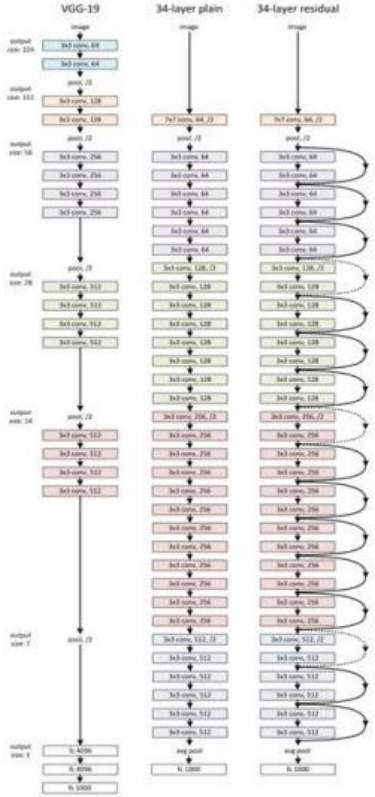


Learn the residual mapping F(x) rather than unreferenced H(x)

Deep Residual Network - ResNet

Design Deep Residual Network

- Keep it simple, just deep
- Design based on VGG style
  - All 3*3 conv (almost)
  - Batch normalization and ReLU
  - Downsampling: cov with stride of 2
  - Spatial size/2 => # filters *2
- No hidden layer, no dropout



Deep Residual Network - ResNet

Experiments: Plain Net

- **Degradation problem**
- Deeper 34-layer plain net has higher validation error than 18 layer plain net
- 34-layer plain net has higher training error through the whole training procedure

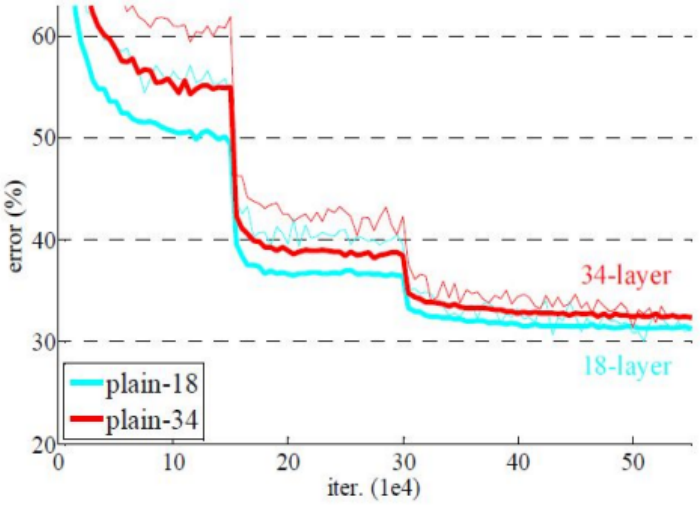


Figure 4: Training on Image Net

Deep Residual Network - ResNet

Experiments: ResNet

34-Layer ResNet is better than 18-layer ResNet (**by 2.8%**)

- lower training error
- **Degradation problem is addressed**

34-layer ResNet reduced top-1 error by 3.5 percent compared to Plain Net

18-layer plain/residual nets are comparably accurate, but…

- 18-layer Resnet converges faster
- ResNet eases optimization by providing faster convergences

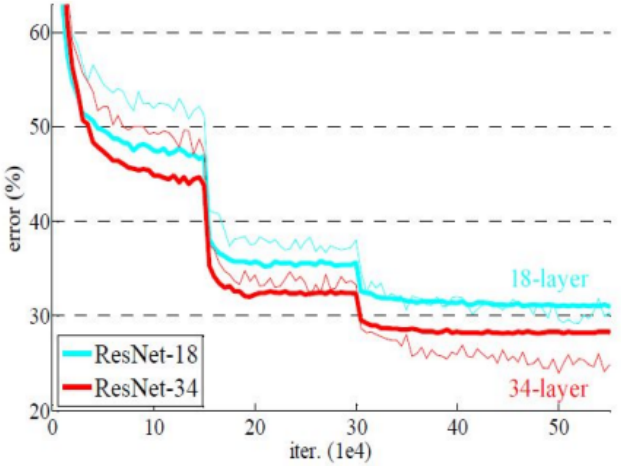


Figure 5: Training on Image Net

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

Table 2: Top-1 Error on ImageNet Validation

Homework2-Applying Transformers to Text Data

1. Transformers for Sentiment Analysis

sentences = [
“I love this movie, it is amazing!”,
“This product is terrible and disappointing.”,
“AI is changing the world rapidly.”,
“The food was okay, not great.”,
“I would never recommend this to anyone.” “I am so happy because it's a warm and sunny day!” “The book was absolutely fantastic, a true masterpiece.”
]

Homework2-Applying Transformers to Text Data

1. Transformers for Sentiment Analysis

Use the tokenizer from BERT, GPT2, and LLaMA

```
from transformers import AutoTokenizer

gpt2_tokenizer = AutoTokenizer.from_pretrained("gpt2")

bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

llama_tokenizer = AutoTokenizer.from_pretrained("hf-internal-testing/llama-tokenizer")
```

Homework2-Applying Transformers to Text Data

1. Transformers for Sentiment Analysis

Tokenize the 5 sentences using the 3 tokenizers.

Report the tokenization results